Orchestrating Production-Ready AI Workflows with Apache Airflow (Webinar Mar 5th)

Architecting Agentic MLOps: a Layered Protocol Strategy with A2A and MCP

Key Takeaways

- Robust, interoperable agent automation systems can be engineered by layering an Model Context Protocol (MCP) with Agent-To-Agent (A2A). These protocols can be used for automating an MLOps workflow using agents.
- A2A provides the communication bus, and MCP acts as a universal language for agent capabilities. A layered agent-based approach leads to more extensible systems, where new capabilities can be added without changing the core communication logic in the agentic era.
- The power of a layered agent architecture lies in its ability to adapt and evolve. For organizations navigating the complexities of AI, this means moving beyond rigid, monolithic systems to agile, agent-driven operations.
- The reusable template presented in the article used a multi-agent system design approach, providing an architectural pattern for decoupling orchestration logic from execution logic, a principle used in scalability. This pattern provides a deliberate structure for moving beyond simple, monolithic agents toward collaborative systems.
- Using a layered A2A-MCP pattern is not confined to MLOps. Its principles extend across any domain where dynamic collaboration and adaptable access to capabilities are crucial for building the next generation of intelligent systems, enabling AI agents to move from isolated tasks to coordinated intelligence and unlocking unprecedented levels of automation and adaptability.

Introduction

As the software industry enters the agentic era, developers and architects face a familiar challenge. Just as the rise of microservices required standardized communication patterns, such as REST and gRPC, the proliferation of specialized AI agents requires a robust framework for them to discover, communicate, and collaborate effectively.

This article proposes an architectural pattern that combines two emerging standards: the Agent-to-Agent (A2A) protocol and the Model Context Protocol (MCP). By layering these protocols, we can create robust, scalable, extensible, and interoperable multi-agent systems, where new capabilities can be added without changing the core communication logic in the agentic era.

In this article, we will first introduce the core concepts of each protocol, then apply the layered protocol strategy to an MLOps use case where the goal is to deploy the model if validations succeed, and then walk through the corresponding code that brings it to life. The code will demonstrate an architectural pattern for decoupling orchestration logic from execution logic, a principle used in scalability.

In an agent-driven paradigm, the goal is to replace rigid pipelines with a dynamic team of specialized AI agents. For example, in our use case of MLOps, an Orchestrator agent tasked with deploying a model might need to collaborate with a Validation agent and a Deployment agent. This scenario presents two fundamental challenges: how do these agents discover and communicate with one another, and how do they access the specific tools and data needed for their tasks? The architecture presented addresses this by assigning distinct roles to each protocol.

A2A provides the communication bus, allowing the Orchestrator to find and task the appropriate specialist without hard-coded connections. MCP acts as a universal language for capabilities, ensuring that once tasked, an agent can discover and utilize the necessary tools regardless of its underlying implementation.

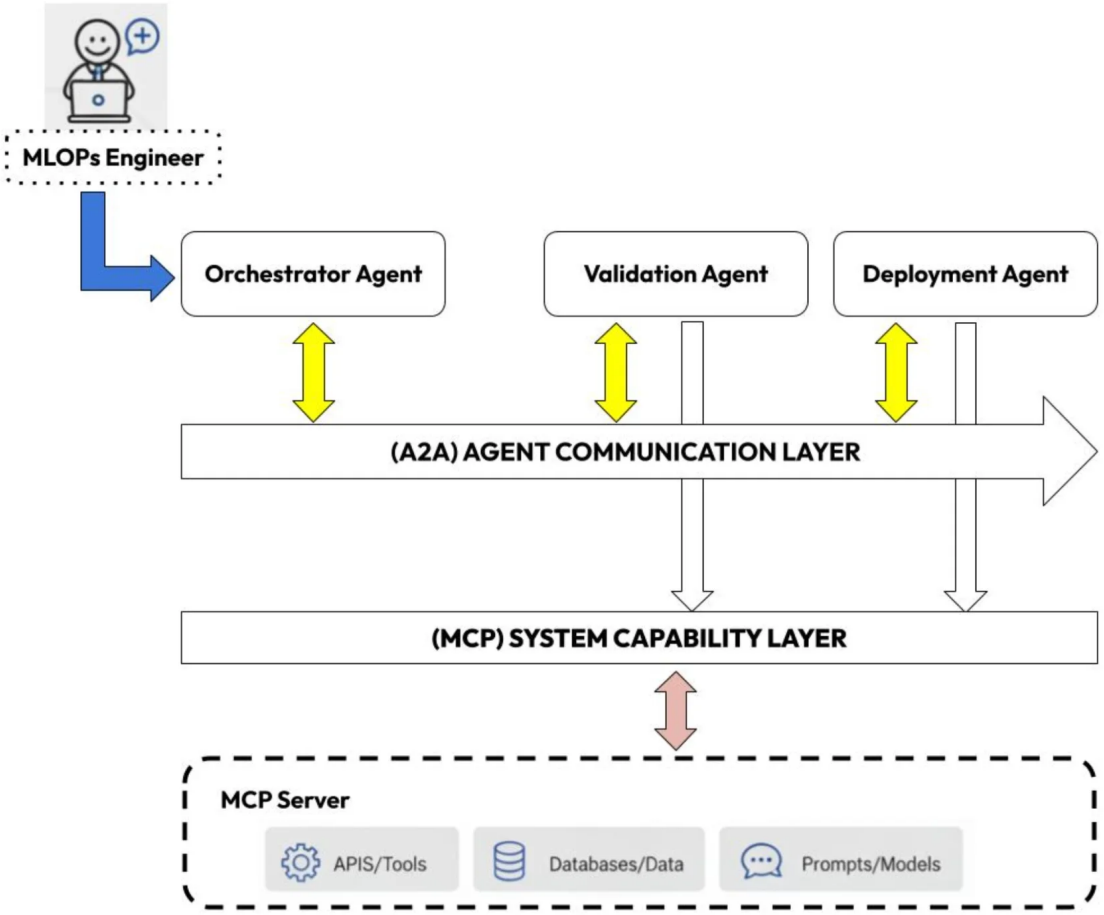


Figure 1: A2A and MCP Stack for Our MLOps Use Case (Source: author)

The example of MLOps use case is chosen deliberately to serve as a conceptual bridge, illustrating the evolution from today's static pipelines to tomorrow's dynamic, agent-driven operations. While existing orchestrators are powerful, their rigidity can become a future bottleneck. When business logic changes, pipelines often need to be rewritten and redeployed. The layered agent architecture, by contrast, is built for this kind of evolution. Our demonstration of an Orchestrator coordinating Validation and Deployment agents will highlight this key advantage: the ability to adapt to new requirements by composing capabilities, not by rewriting huge swaths of code. This shift from static execution to dynamic coordination is the core principle we aim to demonstrate, as AI Agents evolve.

The principles shown here are not limited to MLOps and can be applied to any domain.

Layered Protocols in Our Workflow

A2A: The Agent-to-Agent Communication Bus

A2A is designed to enable AI agents, regardless of the vendor, to securely communicate across different systems. It addresses the need for interoperability in multi-agent environments. By allowing agents from various vendors to interoperate, A2A helps unlock modular workflows, reduces vendor lock-in, and enhances scalability. Think of this as one common language for your Agents.

Key Mechanics

- **Key Element for Interoperability**
  In an A2A world, each agent is assigned an "Agent Card" describing its capabilities, supported protocols, and acceptable request types, enabling other agents to discover and interact without exposing sensitive details. Think of this as the traits of the agent. As your agent evolves, so does this card, allowing the external world to recognize the upgrades.
- **Communication**
  In A2A, messages are exchanged over standard web technologies using formats like JSON and JSON-RPC. This simplifies integration with existing web infrastructures, which is essential because the arrival of agents should not disrupt existing communication technologies.
- **Security & Governance**
  A2A has been brought under the Linux Foundation to promote neutral, collaborative governance and long-term sustainability.

Why A2A matters:

- Turns isolated "single-shot LLM tools" into multi-agent systems capable of cooperation, negotiation, and specialization.
- Enables workflows where one agent can call another as a peer, not just as an API client.
- Supports scaling intelligence horizontally: instead of building one monolithic agent, you orchestrate ecosystems of smaller, specialized ones.

MCP: The Domain-Specific Language

MCP is a protocol intended to standardize how AI systems connect to tools, services, and data sources. Often described as the "USB-C of AI integrations," MCP provides a universal interface allowing AI applications to plug into external data sources and tools without custom glue code.

Key Mechanics

- **Key Element for Interoperability**
  MCP servers expose three primary types of entities. Tools provide actions the agent can invoke, like executing code or calling APIs. Resources include structured data that agents can query or load. Prompts provide predefined templates to guide agent behavior. These primitives are standard definitions so that any MCP-compatible client can discover and use them without custom integrations.
- **Communication**
  Similar to A2A, MCP tries to reuse the existing communication technologies like HTTP, SSE, etc. It also uses a simple Client-Server Architecture.
- **Security & Governance**
  MCP enables powerful integrations, but also introduces risks such as prompt injection, tool poisoning, and unauthorized data access. Although using it alone may not be ideal, it can be effectively bundled with other tools like MCPWatch to enhance system protection.

Why MCP Matters:

MCP enables agents to move beyond fixed skills, allowing them to discover and use any available tool or resource on the network. This supports adding new capabilities without rebuilding agents.

MCP allows seamless tool integration by letting agents treat tools as discoverable services. This eliminates custom integration logic and simplifies adding new functions, APIs, or datasets.

MLOps Workflow

To demonstrate our layered architecture, we will use a very common MLOps workflow use case where we automate the validation and deployment of a machine learning model. The system is composed of three specialized agents that collaborate to achieve the goal:

- **Orchestrator Agent**
  Acts as the coordinator. It translates a high-level goal (e.g., "validate and deploy the latest model") into a sequence of tasks. Using the A2A protocol, it discovers the appropriate specialist agent for each task, passes the required context, and makes decisions based on the results.
- **Validation Agent**
  A specialist agent focused on model validation. It exposes its capabilities, such as performance testing or bias analysis, via its A2A Agent Card. To execute a request, it discovers and uses the underlying MCP tools that implement these checks. This allows the Orchestrator to request a validation without needing to know the implementation details.
- **Deployment Agent**
  A specialist agent responsible for deploying a validated model. Like the Validation Agent, it uses its A2A card to advertise its capabilities and discovers the necessary MCP tools to perform the deployment.

Sequence Diagram for the workflow:

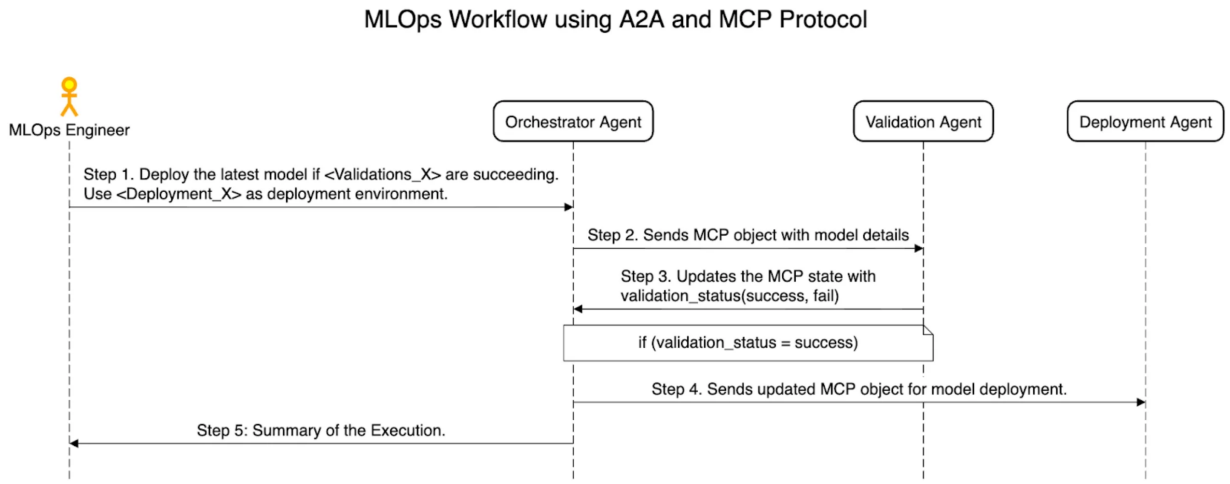


Figure 2: Sequence Diagram for the MLOps Workflow (Source: author)

Execution and flow

From Query to Orchestration

The process begins when the MLOps engineer submits the high-level query. The `OrchestratorAgent` receives this query in its stream method. It immediately calls its `internal _create_plan_from_query` method, using its LLM-driven reasoning to break the complex request into a `TaskList` of two distinct, high-level sub-goals: one for validation and one for deployment.

From Orchestration to Specialization

The Orchestrator's stream method begins executing the plan. For the first task, it uses A2A to discover and call the `ValidationAgent`, passing it the specific validation instruction. The `ValidationAgent` now receives this sub-query in its stream method. It then calls the `_create_tool_use_plan` method. This is an important distinction: its plan is not about delegating, but about using tools. It discovers the `fetch_model` and `validate_churn_model` tools via MCP and formulates a sequence of tool calls to satisfy the request. How it does that should be encoded in its `prompt_personality` string defined at initialization.

From Tools to Results

The ValidationAgent executes its tool-use plan, calling the MCP server to get the job done and streaming the results back to the Orchestrator. If the validation is successful, the Orchestrator proceeds to the second task, calling the `DeploymentAgent`. The `DeploymentAgent` follows the same pattern: it creates a tool-use plan (first fetching the current state, then deploying) and executes it. The final result is then streamed back to the user.

Code Pattern Walkthrough

We will now translate our architectural theory into practice. For this code walkthrough, we will focus on an example query from an MLOps engineer:

> *"Retrieve the latest churn prediction model and run it through the validation module. If the model's absolute bias is less than or equal to 0.04, approve it for deployment. Deploy the new model to the alternate region: if the current production model is running in us-west-1, deploy this version to us-west-2; otherwise, deploy it to us-west-1".*

To build a system capable of executing such commands, we will first establish its foundational components. We will start by setting up the MCP server, which acts as the bridge between the agents and the underlying tools they need to perform their tasks. Then we will present A2A Building blocks and also the glue that connects both the protocols.

A Note on Implementation*: Many functions in the code are intentionally left as placeholders. This is because their internal logic is specific to the implementation (e.g., the choice of validation libraries, cloud vendors, or deployment tools). Our focus in this article is on presenting the architectural pattern for how these components interact.*

MCP Server

The MCP server acts as the central hub for all capabilities in our system. It provides a standardized and discoverable interface to the tools and resources that the specialist agents will use to perform their tasks. This server decouples the agents from the underlying application logic.

For our MLOps workflow, the MCP server exposes the following key endpoints:
Tools (Actions the agents can invoke)

- `fetch_model`
  : Retrieves the metadata for the latest trained model from a model registry.
- `validate_churn_model`
  : Executes the validation logic against a model based on the provided requirements.
- `deploy_churn_model`
  : Triggers the deployment of a validated model to a specific environment.

Resources (Structured data the agents can query)

- `list_agent_cards`
  : Provides a list of all available agents in the system.
- `retrieve_agent_skills`
  : Fetches the detailed capabilities of a specific agent.

The Python code below demonstrates how this server and its endpoints are defined using the FastMCP library. Note that the internal implementation logic within each function is intentionally omitted, as this would vary based on other data tools. The focus here is on the architectural pattern: how capabilities are defined, named, and exposed through a standardized protocol, making them available for any authorized agent to use.

```python
#mcp_server.py
from mcp.server.fastmcp import FastMCP

def serve(host, port, transport):
    """Initializes and runs the MCP Server

    Args:
        host: The hostname or IP address to bind the server to.
        port: The port number to bind the server to.
        transport: The transport mechanism for the MCP server (e.g
    """
    mcp = FastMCP("validation-deployment-mcp-server", host=host, p

    @mcp.tool(
        name="fetch_model",
        description="MCP Tool that fetches the latest trained user
    )
    def fetch_model(model_version_metadata : dict) -> dict:
        """MCP Tool that fetches the latest trained user churn mod
        Args:
            model_version_metadata: Which model data is required.

        Returns:
            JSON object that returns the Metadata information wher
            is present and other metadata for validation purposes
            for validation etc.
        """
        pass

    @mcp.tool(
        name="validate_churn_model",
        description="MCP that validates the churn model.",
    )
    def validate_churn_model(validation_config: dict) -> dict:
        """MCP Tool that validates the churn model based on valida

        Args:
```

```
        validation_config: config containing validation requir

    Returns:
        JSON object returning the validation status.
    """
    pass

@mcp.tool(
    name="deploy_churn_model",
    description="MCP that deploys the churn model.",
)
def deploy_churn_model(deployment_config: dict) -> dict:
    """MCP Tool that deploys the churn model based on deployme

    Args:
        deployment_config: config containing deployment requir

    Returns:
        JSON object returning the deployment status.
    """
    pass

@mcp.resource("resource://list_agent_cards/list", mime_type="a
def list_agent_cards() -> dict:
    """Retrieves all loaded agent cards as a json / dictionary

    This function serves as the handler for the MCP resource i
    the URI 'resource://agent_cards/list'.

    Returns:
        A JSON object containing a list of all available agent
    """

@mcp.resource(
    "resource://retrieve_agent_skills/{agent_name}", mime_type
)
def retrieve_agent_skills(agent_name: str) -> dict:
    """Retrieves an agent card as JSON data.
```

```
        Returns:
            A JSON object of Agent Card.
        """
        pass

    mcp.run(transport=transport)


def main(host, port, transport) -> None:
    serve(host, port, transport)
```

MCP Client

For an agent to discover and use the capabilities exposed by the MCP server, it needs a client. This client module acts as a high-level API that abstracts away the raw details of the MCP protocol. Instead of forcing each agent to construct resource URIs and manage connection states, it provides a clean, reusable interface with methods like `list_agents()` and `list_tools()`.

The code below outlines a simple `MCPClient` class built around the `mcp.ClientSession`. It uses an asynchronous context manager to handle the lifecycle of the connection to the server. Note that the connection details are simplified to highlight the API design rather than the full implementation of a transport-specific connection.

```
from contextlib import asynccontextmanager
from typing import Any, AsyncGenerator, Dict, List

from mcp import ClientSession
from mcp.types import ReadResourceResult, ListResourcesResult, Li


class MCPClient:
    """A high-level client for interacting with the MLOps MCP serv

    def __init__(self, host: str, port: int, transport: str):
        """
```

```
        Initializes the client with the server's connection detai

        Args:
            host: The hostname or IP of the MCP server.
            port: The port of the MCP server.
            transport: The transport mechanism (e.g., 'http', 'sse
        """
        self._host = host
        self._port = port
        self._transport = transport

    @asynccontextmanager
    async def _get_session(self) -> AsyncGenerator[ClientSession,
        """
        Provides a managed session to connect with the MCP server.
        The actual implementation of this would depend on the chos
        """
        # In a real implementation, you would initialize the sessi
        # based on self._host, self._port, etc.
        session: ClientSession = None  # Placeholder for the actua
        try:
            # For example:connected through http
            yield session
        finally:
            # For example: await session.close()
            pass

    async def list_agents(self) -> ReadResourceResult:
        """
        Retrieves the list of all available agent cards from the M
        """
        async with self._get_session() as session:
            return await session.read_resource("resource://list_ag

    async def get_agent_skills(self, agent_name: str) -> ReadResou
        """
        Retrieves the skills for a specific agent from the MCP ser
        """
```

```
        async with self._get_session() as session:
            uri = f"resource://retrieve_agent_skills/{agent_name}'
            return await session.read_resource(uri)

    async def list_resources(self) -> ListResourcesResult:
        """Lists all available resources on the MCP server."""
        async with self._get_session() as session:
            return await session.list_resources()

    async def list_tools(self) -> ListToolsResult:
        """Lists all available tools on the MCP server."""
        async with self._get_session() as session:
            return await session.list_tools()
```

Execution Helpers for Agents

To execute a multi-step plan, an agent needs a structured way to manage its tasks. The following helper classes provide a reusable pattern for this task. The core idea is to represent a complex goal as a TaskList, which is essentially a plan or a sequence of individual Task objects. Each Task represents a single, concrete step in the workflow, such as finding the right specialist agent or invoking a specific tool.

This approach allows the agent's high-level reasoning to be decoupled from the low-level execution mechanics.

```
import json
from collections.abc import AsyncIterable
from a2a.client import A2AClient
from uuid import uuid4
import httpx
from a2a.types import (
    AgentCard,
    MessageSendParams,
    SendStreamingMessageRequest,
    SendStreamingMessageSuccessResponse,
    TaskArtifactUpdateEvent,
)
```

```
from mcp_client import MCPClient
from a2a.server.agent_execution import AgentExecutor, RequestConte
from a2a.server.events import EventQueue


class Task:
    """Represents a single task that needs to be executed in the t

    task_query: str

    def __init__(self, *args, **kwargs):
        pass

    async def find_agent_for_task(self, mcp_client, query) -> Agen
        """Fetch an agent card suitable for the node's task from M

        result = await mcp_client.list_agents(query)
        chosen_agent = select_agent(query)
        agent_card_json = json.loads(chosen_agent.content[0].text)

        return AgentCard(**agent_card_json)

    async def execute_task(
        self,
    ) -> AsyncIterable[dict[str, any]]:
        """Execute the node task via A2A streaming messages using
        agent_card = await self.find_agent_for_task(query=self.tas
        async with httpx.AsyncClient() as httpx_client:
            client = A2AClient(httpx_client, agent_card)  # A2A Cl
            payload: dict[str, any] = {
                "message": {
                    "parts": [{"kind": "text", "text": self.task_d
                    # Can have other elements too based on Agent C
                },
            }
            request = SendStreamingMessageRequest(
                id=str(uuid4()), params=MessageSendParams(**paylo
            )
```

```
            response_stream = client.send_message_streaming(reques
            async for chunk in response_stream:
                # Save the artifact as a result of the node
                if isinstance(chunk.root, SendStreamingMessageSuc
                   isinstance(chunk.root.result, TaskArtifactUpdat
                    artifact = chunk.root.result.artifact
                    self.results = artifact
                yield chunk


class TaskList:
    """Represents a Topological graph of tasks that need to be exe

    task_list: list[Task]  # Task list that needs to be executed.

    def __init__(self, *args, **kwargs) -> None:
        """
        Breaks the query into a task list and the order in which i
        executed.The AI agent should break this down and put it in
        array.
        """
        pass

    async def execute_task_list(self) -> AsyncIterable[dict[str, a
        """
        Executes the tasks for the agent.
        """
        # .....
        for task in self.task_list:
            # ....
            task.execute_task()



class GenericAgentExecutor(AgentExecutor):
    """AgentExecutor used by the agents."""

    def __init__(self, agent):
```

```
        self.agent = agent

    async def execute(
        self,
        context: RequestContext,
        event_queue: EventQueue,
    ) -> None:
        pass
```

Orchestrator Agent

Card

```
{
    "name": "Orchestrator Agent",
    "description": "Helps in invoking the MLOps workflow. Which wi
    "url": "http://localhost:8003/",
    "version": "1.0.0",
    "skills": [
        {
            "id": "orchestrate_the_flow",
            "name": "orchestrate_the_flow",
            "description": "Helps in orchestrating MLOps Workflow'
            "tags": [
                "Validate the model and then deploy it."
            ],
            "examples": [
                "Retrieve the latest churn prediction model and ru
            ]
        }
    ]
}
```

Code Boilerplate

```
from typing import AsyncIterable, Any
```

```python
from agent_helpers import TaskList
from mcp_client import MCPClient

class OrchestratorAgent:
    """
    Orchestrates a multi-step workflow by breaking a high-level go
    into a sequence of tasks for specialist agents.
    """

    def __init__(self, mcp_client: MCPClient, prompt_personality:
        """
        Initializes the Orchestrator Agent.

        Args:
            mcp_client: A client for interacting with the MCP serv
            prompt_personality: Instructions guiding the agent's p
        """
        self._mcp_client = mcp_client
        self._prompt_personality = prompt_personality

    async def _create_plan_from_query(self, query: str) -> TaskLis
        """
        Translates a natural language query into a structured Task
        """
        # This method simulates the agent's high-level reasoning p
        # The agent's LLM, guided by its personality prompt, woulc
        # user's query to identify distinct, sequential steps.

        # For our example query, it would identify two main sub-go
        # 1. A validation step with a specific condition.
        # 2. A deployment step that depends on the outcome of the

        # The agent then creates a TaskList where each Task encaps
        # natural language instruction for that sub-goal. This is
        # from a specialist agent, whose plan would involve speci
        #
        # Task 1 Query: "Retrieve the latest churn prediction mode
        # Task 2 Query: "Deploy the new model to the alternate reg
```

```
        #
        # The output of this method would be a TaskList object cor
        # these two Task objects, ready for execution.
        pass

    async def stream(self, query: str) -> AsyncIterable[dict[str,
        """
        Processes a query by creating a plan and then executing it
        """
        # 1. CREATE THE PLAN
        # The agent first calls its internal planning method to tr
        # the natural language query into a structured TaskList.
        plan = await self._create_plan_from_query(query)

        # 2. EXECUTE THE PLAN
        # The agent then executes the plan. The plan.execute() met
        # iterate through the Tasks. For each Task, it will find t
        # appropriate specialist agent (Validation, then Deploymer
        # stream the sub-query to it. The results are then yielded
        pass
```

Validation Agent

Card

```
{
    "name": "Validation Agent",
    "description": "Helps in validating the MLOps model.",
    "url": "http://localhost:8004/",
    "version": "1.0.0",
    "skills": [
        {
            "id": "validate_the_model",
            "name": "validate_the_model",
            "description": "Helps in validating MLOps models",
            "tags": [
                "Validate the model based on user requirements."
```

```
            ],
            "examples": [
                "Retrieve the latest churn prediction model and ru
            ]
        }
    ]
}
```

Code Boilerplate

```
from typing import AsyncIterable, Any
from mcp_client import MCPClient

class ValidationAgent:
    """
    A specialist agent that validates a machine learning model by
    and using tools from the MCP server.
    """

    def __init__(self, mcp_client: MCPClient, prompt_personality:
        """
        Initializes the Validation Agent.

        Args:
            mcp_client: A client for interacting with the MCP serv
            prompt_personality: Instructions guiding the agent's t
        """
        self._mcp_client = mcp_client
        self._prompt_personality = prompt_personality

    async def _create_tool_use_plan(self, query: str):
        """
        Translates a natural language query into a structured plar
        """
        # This method simulates the agent's reasoning process.

        # 1. DISCOVER: The agent first needs to understand what it
```

```
        # It would call self._mcp_client.list_tools() to get a rea
        # list of all available capabilities on the MCP server. Th
        # it to dynamically learn that tools like 'fetch_model' ar
        # 'validate_churn_model' are available. This should be par
        # prompt_personality

        # 2. PLAN: Based on the available tools and the specific u
        # the agent formulates a plan. For the query: "...absolute
        # less than or equal to 0.04...", its LLM would determine
        # needs to:
        #   a. Fetch the model's metadata using the 'fetch_model'
        #   b. Construct a 'validation_config' containing the bias
        #      extracting the '0.04' threshold from the query.
        #   c. Call the 'validate_churn_model' tool with that conf
        #
        # The output of this method would be a structured object,
        # of pre-configured tool calls, ready for execution.
        pass

    async def stream(self, query: str) -> AsyncIterable[dict[str,
        """
        Processes a validation query by creating a plan and then e
        """
        # 1. CREATE THE PLAN
        # The agent first calls its internal planning method to tr
        # the natural language query into a structured sequence of
        plan = await self._create_tool_use_plan(query)

        # 2. EXECUTE THE PLAN
        # The agent would then iterate through the steps in the ge
        # It would call the necessary MCP client methods (fetch_mo
        # validate_churn_model) in the correct order with the corr
        # parameters derived during the planning phase. The result
        # step would be yielded back to the Orchestrator.
        pass
```

Deployment Agent

Card

```
{
    "name": "Deployment Agent",
    "description": "Helps in deploying the validated MLOps model.'
    "url": "http://localhost:8005/",
    "version": "1.0.0",
    "skills": [
        {
            "id": "deploy_the_model",
            "name": "deploy_the_model",
            "description": "Helps in deploying MLOps models",
            "tags": [
                "Deploy the model based on user requirements."
            ],
            "examples": [
                "Deploy the new model to the alternate region: if
            ]
        }
    ]
}
```

Code Boilerplate

```
from typing import AsyncIterable, Any
from mcp_client import MCPClient

class DeploymentAgent:
    """
    A specialist agent that deploys a validated machine learning m
    discovering and using tools from the MCP server.
    """

    def __init__(self, mcp_client: MCPClient, prompt_personality:
        """
        Initializes the Deployment Agent.
```

```
        Args:
            mcp_client: A client for interacting with the MCP serv
            prompt_personality: Instructions guiding the agent's t
        """
        self._mcp_client = mcp_client
        self._prompt_personality = prompt_personality

    async def _create_tool_use_plan(self, query: str):
        """
        Translates a natural language query into a structured plar
        """
        # This method simulates the agent's reasoning process.

        # 1. DISCOVER: The agent determines its available capabili
        # It would call self._mcp_client.list_tools() to learn tha
        # like 'fetch_model' and 'deploy_churn_model' are availabl
        # personality.

        # 2. PLAN: The agent formulates a plan based on the query:
        # the new model to the alternate region...". Its LLM reasc
        #   a. To find the "alternate" region, I must first find t
        #   b. The 'fetch_model' tool can get me the metadata of t
        #      production model.
        #   c. From that metadata, I can extract the current deplc
        #   d. I can then write logic to determine the alternate r
        #   e. The final plan is a sequence of two tool calls: fir
        #      to get the state, then deploy_churn_model to execut

        # The output of this method would be a structured object,
        # of pre-configured tool calls, ready for execution.
        pass

    async def stream(self, query: str) -> AsyncIterable[dict[str,
        """
        Processes a deployment query by creating a plan and then e
        """
        # 1. CREATE THE PLAN
```

```
        # The agent first calls its internal planning method to tr
        # the natural language query into a structured sequence of
        plan = await self._create_tool_use_plan(query)

        # 2. EXECUTE THE PLAN
        # The agent would then iterate through the steps in the ge
        # It would call the necessary MCP client methods (fetch_mo
        # deploy_churn_model) in the correct order with the correc
        # parameters derived during the planning phase. The result
        # step would be yielded back to the caller.
        pass
```

Script to start all the Agents

```
import json
import httpx
from pathlib import Path
from basic_helper.promp_personalities import prompts
from orchestrator_agent import OrchestratorAgent
from validation_agent import ValidationAgent
from deployment_agent import DeploymentAgent
from a2a.types import AgentCard
import uvicorn
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import (
    BasePushNotificationSender,
    InMemoryPushNotificationConfigStore,
    InMemoryTaskStore,
)

mcp_client = MCPClient(host="localhost", port=8000, transport="htt

def get_agent(agent_card: AgentCard):
    """Get the agent, given an agent card."""
    try:
```

```
        if agent_card.name == "Orchestrator Agent":
            # This is the Orchestrator Agent
            return OrchestratorAgent(mcp_client, prompts.orchestra
        if agent_card.name == "Validation Agent":
            # This is the Validation Agent
            return ValidationAgent(mcp_client, prompts.validation_
        if agent_card.name == "Deployment Agent":
            # This is the Deployment Agent
            return DeploymentAgent(mcp_client, prompts.deployment_

    except Exception as e:
        raise e

def main(host, port, agent_card_path):
    """Starts an Agent server."""

    with Path.open(agent_card) as file:
        data = json.load(file)
    agent_card = AgentCard(**data)

    client = httpx.AsyncClient()
    push_notification_config_store = InMemoryPushNotificationConf
    push_notification_sender = BasePushNotificationSender(
        client, config_store=push_notification_config_store
    )

    request_handler = DefaultRequestHandler(
        agent_executor=GenericAgentExecutor(agent=get_agent(agent_
        task_store=InMemoryTaskStore(),
        push_config_store=push_notification_config_store,
        push_sender=push_notification_sender,
    )

    server = A2AStarletteApplication(
        agent_card=agent_card, http_handler=request_handler
    )

    uvicorn.run(server.build(), host=host, port=port)
```

```
if __name__ == "__main__":
    main()
```

Architectural Benefits of Layering those two protocols

This clear separation of orchestration from specialized execution yields significant architectural benefits:

- **Dynamic Discovery and Resilience**
  The Orchestrator has no hardcoded knowledge of the specialists. New agents (e.g., a ReportingAgent or a MonitoringAgent) can be added to the system, and the Orchestrator will be able to discover and use them without any changes to its code.
- **Composable Capabilities**
  The specialist agents themselves are not monolithic. They compose their behavior by discovering and using granular tools from the MCP server. A new validation check can be added simply by deploying a new MCP tool, which the ValidationAgent can then discover and use dynamically.
- **Clear Separation of Intent from Execution**
  The Orchestrator expresses the high-level business goal. The specialists handle the low-level implementation details. This decoupling makes the entire system easier to understand, maintain, and scale.
- **Adaptive and Emergent Systems**
  By combining a generalist coordinator with a discoverable set of specialized tools and agents, we create a system that can adapt to new and complex commands that were not explicitly designed for.

By layering a communication and discovery protocol (A2A) on top of a capability protocol (MCP), we bridge the gap from rigid and procedural automation to truly goal-oriented, AI-driven operations.

Conclusion

As the agentic era introduces a new paradigm for software development, the need for robust, scalable, and interoperable agent systems becomes important. In this article, we have proposed an architectural pattern that leverages the Agent-to-Agent (A2A) and Model Context Protocol (MCP) to address this challenge.

Through the detailed exploration of an MLOps workflow, we demonstrated how this layered approach successfully decouples orchestration logic from execution logic, a fundamental principle for scalable systems. We showed how A2A provides the necessary communication framework for dynamic agent collaboration, while MCP acts as a universal interface for agents to discover and utilize diverse tools and resources. This architecture enables the seamless integration of new capabilities without altering the core communication logic.

The power of this layered agent architecture lies in its ability to adapt and evolve. For organizations navigating the complexities of AI, this means moving beyond rigid, monolithic systems to agile, agent-driven operations. It provides a robust blueprint for developing AI ecosystems that can rapidly incorporate new models, tools, and business requirements. Developers gain a powerful framework to build more resilient and maintainable pipelines. This pattern is not confined to MLOps; its principles extend across any domain where dynamic collaboration and adaptable access to capabilities are crucial for building the next generation of intelligent systems. By embracing A2A and MCP, we enable AI agents to move from isolated tasks to coordinated intelligence, unlocking unprecedented levels of automation and adaptability in the agentic era.

The architectural pattern presented here offers an approach to multi-agent design. It provides a deliberate structure for moving beyond simple, monolithic agents toward collaborative systems.

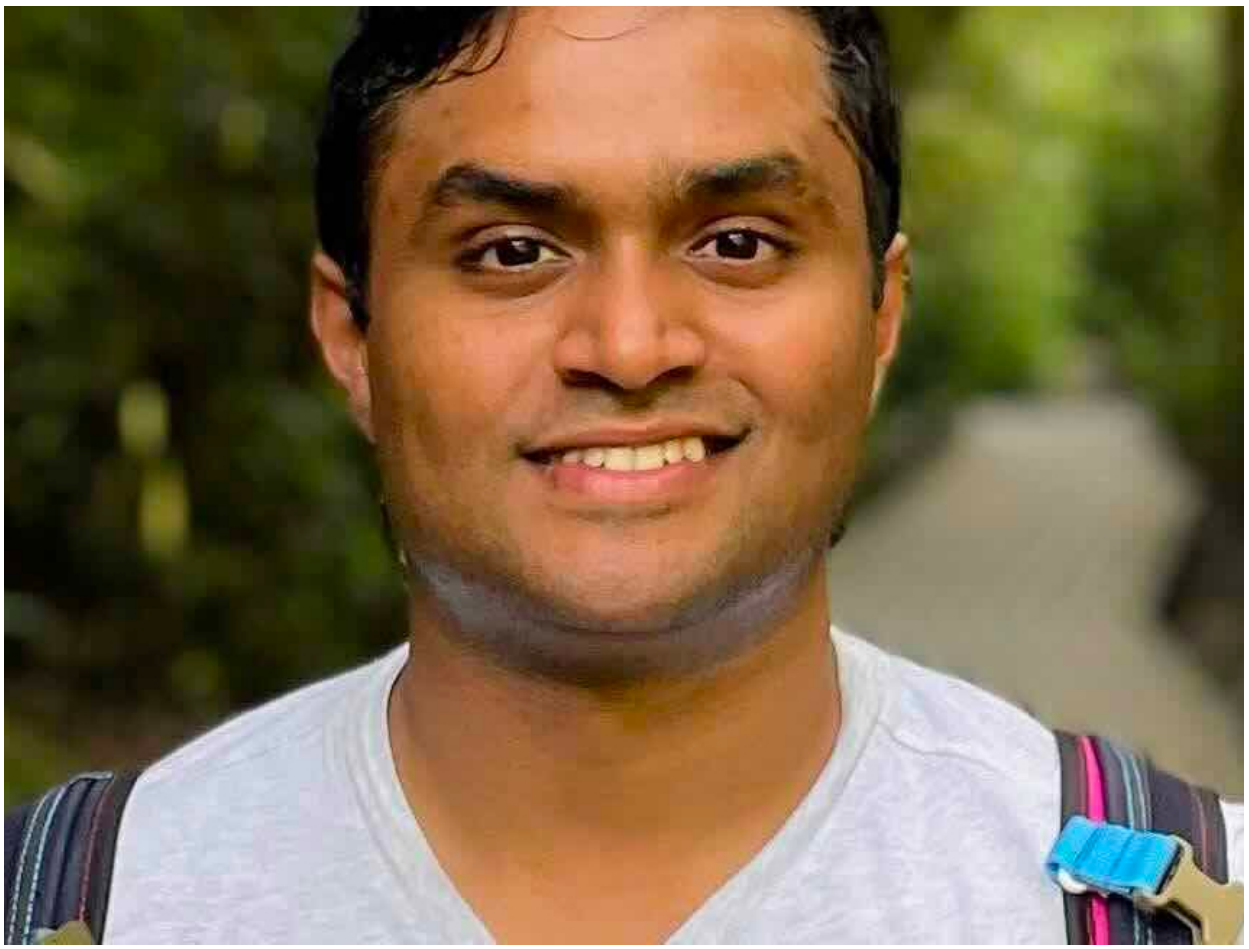
For readers interested in experimenting with these concepts and developing tools around it, the official A2A Samples repository on GitHub provides a runnable example using these two protocols and is an excellent resource for getting started.

About the Authors

Shashank Kapoor

Shashank Kapoor is an ML Engineer in Google Ads, working at the intersection of privacy and machine learning. He leads the development of privacy-safe conversion measurement solutions, ensuring compliance with new privacy laws while simultaneously powering significant revenue for Google. With an extensive background in deploying ML at scale, he previously led ML development for the Finsights product at American Express, recovering millions in revenue. His expertise includes NLP, Multi-Modal models, Data Science, large-scale data processing, and distributed systems.

Show more
Show less



Sanjay Surendranath Girija

Sanjay Surendranath Girija is a Senior Software Engineer at Google, working on large-scale machine learning models for YouTube video recommendations. His areas of expertise include deep learning, transformers-based models, recommender systems, model optimization, adtech and privacy. He was previously a tech lead in Google Ads where he led several efforts for the Store Sales and Bulldog Conversion Modeling teams, developing new models for conversion measurement that resulted in significant revenue for Google. Prior to this, he was leading the inbound anti-spam detection for Yahoo Mail, where he built ML models, algorithms and systems for spam detection and classification. In addition to his professional endeavors, Sanjay is an active contributor to the broader tech community as an open source contributor to Tensorflow and a peer reviewer for top AI and ML conferences.

Show more
Show less

Lakshit Arora

Lakshit Arora is an Engineering Manager, who is currently at Google leading YouTube's growth platform. His areas of expertise include ML, AI, search and discovery, web applications, and general software development. He leads efforts that drive significant user engagement and revenue growth for YouTube. Before his work on YouTube, Lakshit worked on patented algorithms that empower Google Drive Search. Outside of work, Lakshit also serves on the Technical Program Committee for top computer science conferences and journals as well as on the judging committee for top IT industry innovation awards.

Show more
Show less

Please see https://www.infoq.com for the latest version of this information.